SEEM5020 Project
Frequency Estimation Under the Strict Turnstile Model with α-Bounded Deletions

XIAN Haoming

1 Overview

This project implements and evaluates four classical frequency-estimation methods under the strict turnstile model with the L_1 α-bounded deletion property: Misra-Gries, Space-Saving, Count-Min Sketch, and Count-Sketch. The original repository was updated to use the provided network pcap in `data/network/200002091359.dump` as the real-world dataset and to redesign the experiments around an explicit error-space tradeoff instead of only reporting accuracy under one memory configuration.

Throughout the report, I denotes total insertion mass, D denotes total deletion mass, f denotes the final frequency vector, and $\|f\|_1 = \sum_e f_e$ denotes the final L_1 mass. In the unit-update experiments, I and D are also the numbers of positive and negative updates. In the weighted-update experiments, I and D denote total inserted and deleted mass. Under strict turnstile, both settings satisfy

$$\|f\|_1 = I - D.$$

2 Extended Algorithms

The main algorithmic issue is that Misra-Gries and Space-Saving were originally designed for insertion-only streams, while this project requires strict turnstile streams with bounded deletions. The extensions implemented here keep the algorithms modular and directly aligned with the turnstile setting.

2.1 Misra-Gries Extension

The extended Misra-Gries estimator maintains two weighted summaries:

- a *positive summary* for insertions,
- a *negative summary* for deletion magnitudes.

Each positive update $(e, +v)$ is processed by the standard weighted Misra-Gries rule. Each negative update $(e, -v)$ is processed by the deletion summary as a positive weight v. The final estimate is

$$\widehat{f_e} = \max(0, \widehat{f_e^+} - \widehat{f_e^-}).$$

Under the strict turnstile model the true frequency never becomes negative, so clipping negative estimates to zero is valid. The total memory budget is fixed externally. Since the stream construction enforces

$$D \leq \frac{\alpha - 1}{\alpha + 1} I,$$

the corresponding fraction of deletions among all updates satisfies

$$\frac{D}{I+D} \leq \frac{\alpha-1}{2\alpha}.$$

This derived deletion fraction is the quantity used to bias the positive/negative budget split.

2.2 Space-Saving Extension

The Space-Saving extension uses the same dual-summary decomposition, but each half is a weighted Space-Saving summary instead of Misra-Gries. Within each summary:

- an existing key receives additional weight directly,
- a free slot is used when available,
- otherwise the minimum counter is replaced.

This gives stronger heavy-hitter behavior when the final distribution is skewed, but the replacement rule can be fragile when the budget is very small and the stream has broad support.

2.3 Count-Min Sketch Extension

Count-Min Sketch is a linear sketch, so updates of the form (e, Δ) are handled directly by adding Δ to one bucket in each row. For this project, a single sketch is sufficient because the stream is in the *strict turnstile* model: although updates may be negative, the frequency vector remains entrywise nonnegative at every time. Under this assumption, the standard point estimator

$$\widehat{f}_e = \min_{1 \leq j \leq d} C[j, h_j(e)]$$

retains the usual one-sided interpretation. The corrected implementation therefore uses a single Count-Min sketch rather than separate positive and negative sketches. The budget controls the sketch width for a fixed depth.

2.4 Count-Sketch Extension

Count-Sketch natively supports signed updates, so the extension is simpler. Each update changes one signed counter per row, where the sign is determined by a second hash function. The estimate for an item is the median of the signed row estimates, clipped at zero. As with Count-Min Sketch, a single sketch is the correct construction here; deletions do not require a second structure. The role of α is indirect: it changes the stream difficulty, while the budget controls the table width.

2.5 Error-Space Guarantees

This subsection states the asymptotic guarantees that explain the empirical error-space tradeoff. Let B denote the logical budget in bytes, and let

$$\beta_\alpha = \frac{\alpha-1}{\alpha+1},$$

so that the stream generator satisfies $D \leq \beta_\alpha I$. For the two summary-based extensions, let $\rho \in (0,1)$ denote the fraction of the logical budget assigned to the deletion-side summary.

Misra-Gries. Let s_+ and s_- denote the numbers of insertion-side and deletion-side slots. For the classical insertion-only summary, the additive under-estimation error is $O(I/s_+)$. Applying the same argument to the deletion summary gives error $O(D/s_-)$. Hence the extended estimator satisfies

$$|\widehat{f}_e - f_e| \le O\left(\frac{I}{s_+} + \frac{D}{s_-}\right)$$

Using

$$I + D \le \alpha(I - D),$$

we obtain

$$I \le \frac{\alpha+1}{2}(I-D), \qquad D \le \frac{\alpha-1}{2}(I-D).$$

Therefore

$$|\widehat{f}_e - f_e| \le O\left((I-D)\left(\frac{\alpha+1}{2s_+} + \frac{\alpha-1}{2s_-}\right)\right).$$

Since one Misra-Gries slot uses 16 logical bytes, $s_+ \asymp (1-\rho)B/16$ and $s_- \asymp \rho B/16$ for the chosen budget split ρ. Therefore the worst-case additive error decays as $O_\alpha((I-D)/B)$ for fixed split ratio.

Space-Saving. For insertion-only Space-Saving with m counters, the classical additive over-estimation error is at most $O(N/m)$. Applying this separately to the insertion and deletion summaries yields

$$|\widehat{f}_e - f_e| \le O\left(\frac{I}{s_+} + \frac{D}{s_-}\right) \le O\left((I-D)\left(\frac{\alpha+1}{2s_+} + \frac{\alpha-1}{2s_-}\right)\right).$$

Each Space-Saving slot uses 24 logical bytes, so $s_+ \asymp (1-\rho)B/24$ and $s_- \asymp \rho B/24$. Hence Space-Saving also has an inverse-space additive tradeoff of order $O_\alpha((I-D)/B)$, but with a larger constant than Misra-Gries.

Count-Min Sketch. For a Count-Min sketch with width w and depth d, the standard strict-turnstile point-query guarantee is

$$\widehat{f}_e \le f_e + \varepsilon\|f\|_1 \qquad \text{with probability at least } 1-\delta,$$

when $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$. Since $\|f\|_1 = I - D$ and $B = 8dw$, this can be rewritten as

$$|\widehat{f}_e - f_e| \le O\left(\frac{d}{B}(I-D)\right)$$

for fixed failure probability. Thus the Count-Min additive error decreases inversely with logical space.

Count-Sketch. For Count-Sketch with width w and depth d, the classical point-query guarantee bounds the error by the L_2 tail:

$$|\widehat{f}_e - f_e| \le O\left(\frac{\|f_{-e}\|_2}{\sqrt{w}}\right) \qquad \text{with probability at least } 1-\delta,$$

after taking the median over $d = O(\log(1/\delta))$ rows. Since $B = 8dw$, this becomes

$$|\widehat{f}_e - f_e| \le O\left(\|f_{-e}\|_2\sqrt{\frac{d}{B}}\right).$$

Using the coarse inequality $\|f_{-e}\|_2 \le \|f\|_1 = I - D$, we also obtain

$$|\widehat{f}_e - f_e| \le O\left((I-D)\sqrt{\frac{d}{B}}\right).$$

For fixed confidence, the error therefore decays like $B^{-1/2}$ rather than B^{-1}. This slower asymptotic decay is often offset in practice by the stronger dependence on the L_2 tail rather than the full L_1 mass.

Interpretation. The two summary-based methods have worst-case additive error proportional to $(I-D)/B$ up to an α-dependent constant, Count-Min Sketch has additive error proportional to $(I-D)/B$, and Count-Sketch has the sharper tail-sensitive bound $\|f_{-e}\|_2/\sqrt{B}$ together with the coarse corollary $(I-D)/\sqrt{B}$. This predicts the observed pattern in the experiments: summary methods improve rapidly once enough slots are available to hold the dominant items, whereas the sketch methods are more robust when the support is broad or the budget is tight.

3 Experimental Setup

3.1 Datasets

Three datasets were used.

- **Balanced synthetic**: a uniform distribution over 512 items.
- **Skewed synthetic**: a Zipf-like distribution over 512 items with mass proportional to $(r+1)^{-1.15}$.
- **Real-world network trace**: the pcap file `data/network/200002091359.dump`.

For the network trace, `tcpdump` was used to convert packets into directed endpoint-pair keys of the form

`IP|source_endpoint > destination_endpoint`.

The full trace produced 2,096,377 parsable packets and 96,122 unique flow keys. The top 1024 flow keys by packet count were retained as the real-world weighted distribution used by the experiment runner.

3.2 Strict-Turnstile Stream Construction

For both update modes, the generator enforces the α-bounded deletion condition on total update mass:

$$I + D \le \alpha(I - D),$$

which is equivalent to

$$D \le \frac{\alpha - 1}{\alpha + 1} I.$$

Equivalently,

$$\frac{D}{I+D} \le \frac{\alpha - 1}{2\alpha},$$

where the factor 2 appears only when converting the insertion-referenced ratio into a fraction of all updates. Two update modes are used.

- **Unit updates**: every update has magnitude 1, so update counts and update masses coincide.

- **Weighted updates**: insertion magnitudes are drawn from a truncated power-of-two law on $\{1, 2, 4, 8\}$ with probabilities proportional to $8 : 4 : 2 : 1$.

During stream generation:

- insertion items are sampled from the dataset distribution,
- in unit mode, deletion items are sampled randomly from the currently active multiset,
- in weighted mode, a random deletion schedule is sampled from the inserted mass and then interleaved only when the corresponding item is active,
- every deletion is constrained so that no item frequency becomes negative.

The generator uses fixed random seeds for reproducibility and records the realized insertion mass I, deletion mass D, and final mass $I - D$ for every run. This guarantees:

- strict turnstile feasibility at every prefix,
- the L_1 α-bounded deletion property.

3.3 Parameter Grid

- Update-event counts: 10,000, 25,000, 50,000
- Update modes: unit and weighted
- α values: 1.5, 2, 4, 8
- Budget sweep: 1024, 2048, 4096, 8192, 16384 logical bytes
- Seeds: 3 repetitions per setting
- Accuracy metric: average relative error on the exact top-25 final-frequency items by final mass
- Space metric: logical bytes derived from maintained counters/tables, not Python object overhead

Logical space refers to normalized algorithmic memory usage. One stored integer field is counted as 8 bytes, so a sketch with $d \times w$ counters uses $8dw$ bytes, a Misra-Gries slot uses 16 bytes (item id and counter), and a Space-Saving slot uses 24 bytes (item id, counter, and error term). This avoids conflating algorithmic space complexity with Python container overhead.

4 Results

4.1 Tradeoff Figures

The full artifact set now includes separate unit-update and weighted-update plots. To keep the report concise, the figures below show the three unit-update tradeoff plots together with two representative real-world weighted-update figures.

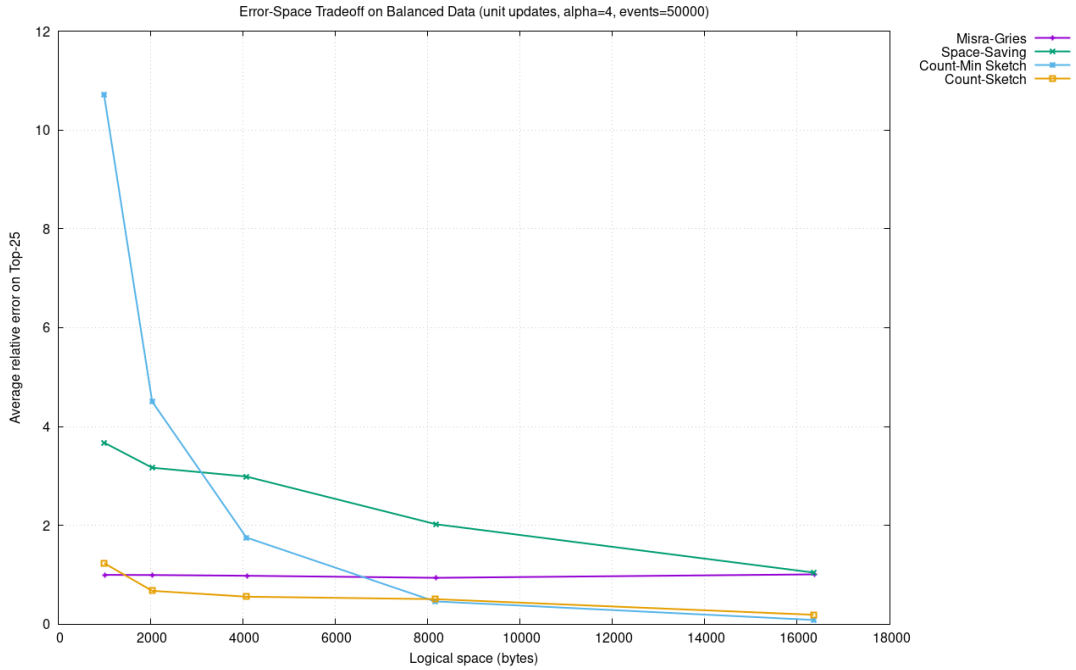

Figure 1: Unit-update error-space tradeoff on balanced synthetic data for $\alpha = 4$ and update-event count 50,000.

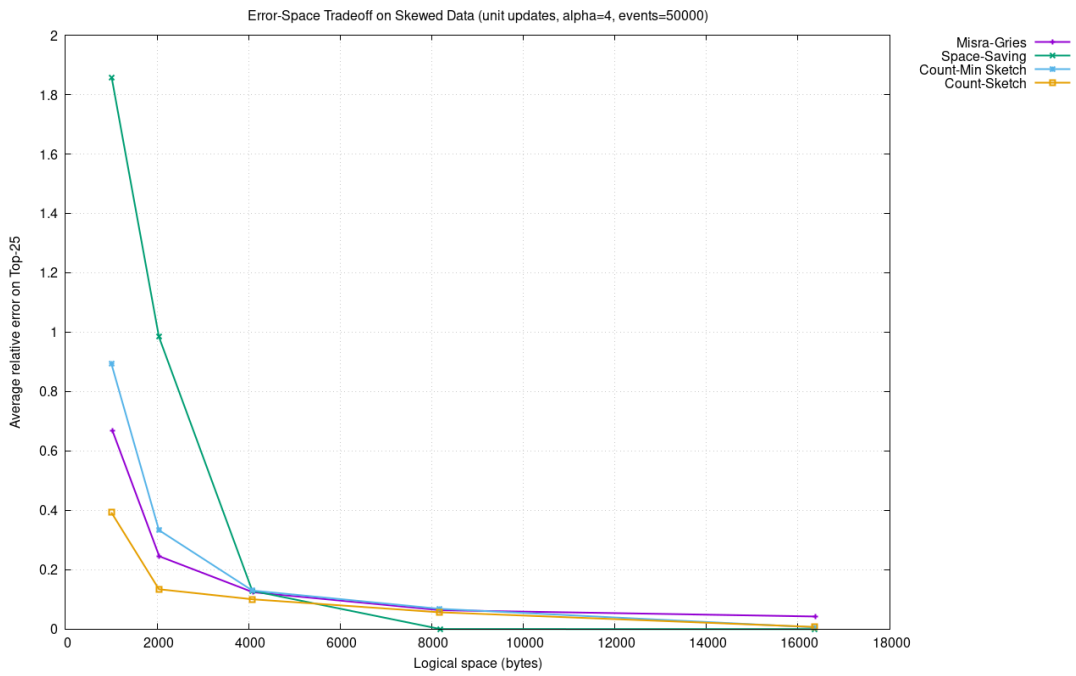

Figure 2: Unit-update error-space tradeoff on skewed synthetic data for $\alpha = 4$ and update-event count 50,000.

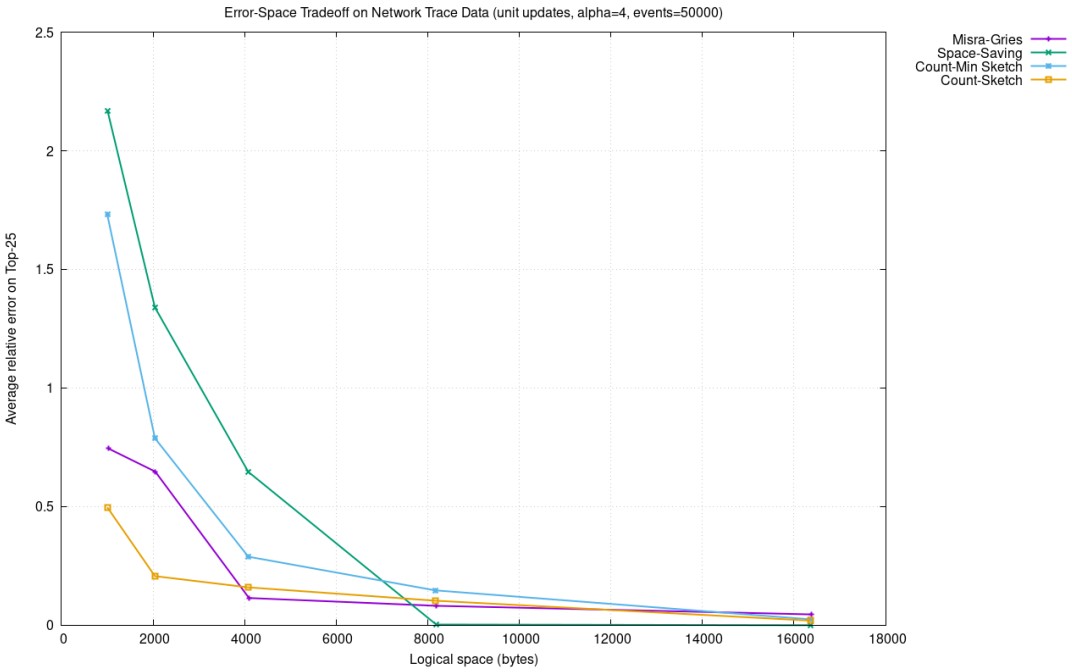

Figure 3: Unit-update error-space tradeoff on the network-trace dataset for $\alpha = 4$ and update-event count 50,000.

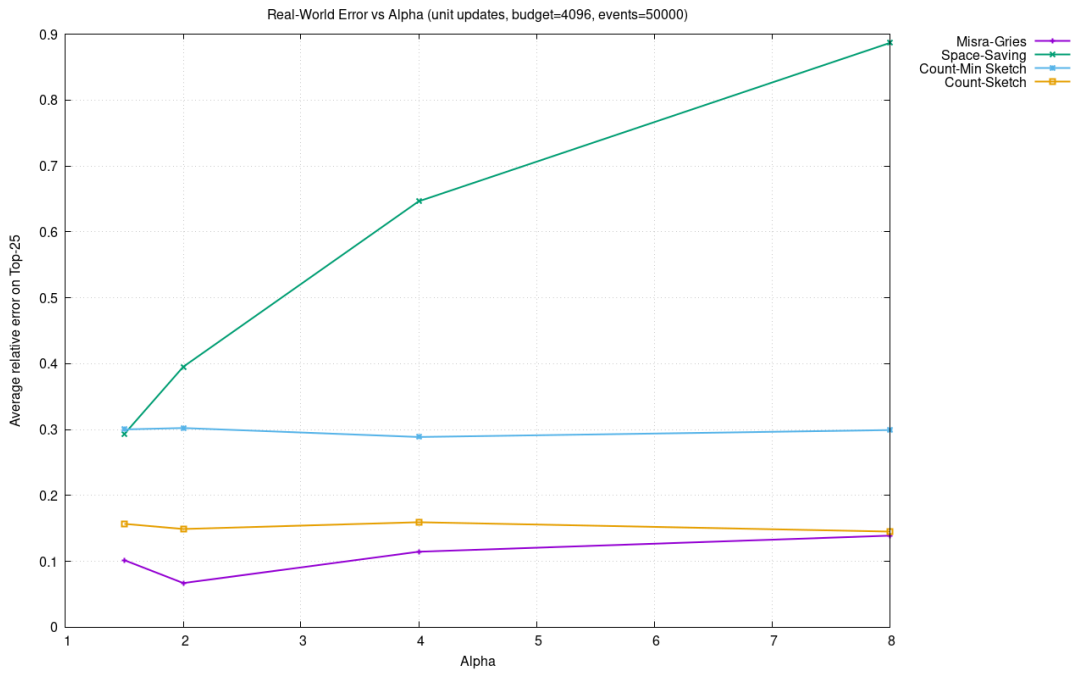

Figure 4: Unit-update real-world accuracy as α changes at fixed 4096-byte budget and update-event count 50,000.

4.2 Representative Tables

Algorithm	1KB	2KB	4KB	8KB	16KB
Misra-Gries	0.7447	0.6473	0.1145	0.0816	0.0454
Space-Saving	2.1663	1.3394	0.6463	0.0028	0.0000
Count-Min Sketch	1.7308	0.7881	0.2886	0.1468	0.0244
Count-Sketch	0.4970	0.2066	0.1593	0.1033	0.0192

Table 1: Unit-update average relative error on the real-world network trace for $\alpha = 4$ and update-event count 50,000 across the memory-budget sweep.

5 Analysis

The regenerated grid exposes a stable tradeoff structure across both unit and weighted updates. Changing from unit to weighted updates changes the realized masses I and D, but it does not overturn the main ranking pattern on the network trace.

Balanced synthetic data remains the hardest workload in both modes. Balanced data has no dominant head, so the summary-based methods have little structure to preserve. At $\alpha = 4$ and update-event count 50,000, Count-Sketch is best through the middle of the budget range in both modes: 0.6750, 0.5567, and 0.5068 at 2KB, 4KB, and 8KB for unit updates, and 0.7137, 0.5261, and 0.4243 for weighted updates. At the highest budget, Count-Min Sketch becomes strongest, reaching 0.0850 in unit mode and 0.1017 in weighted mode at 16KB. Misra-Gries is only the least-bad method at 1KB, but its error remains close to 1 on this workload.

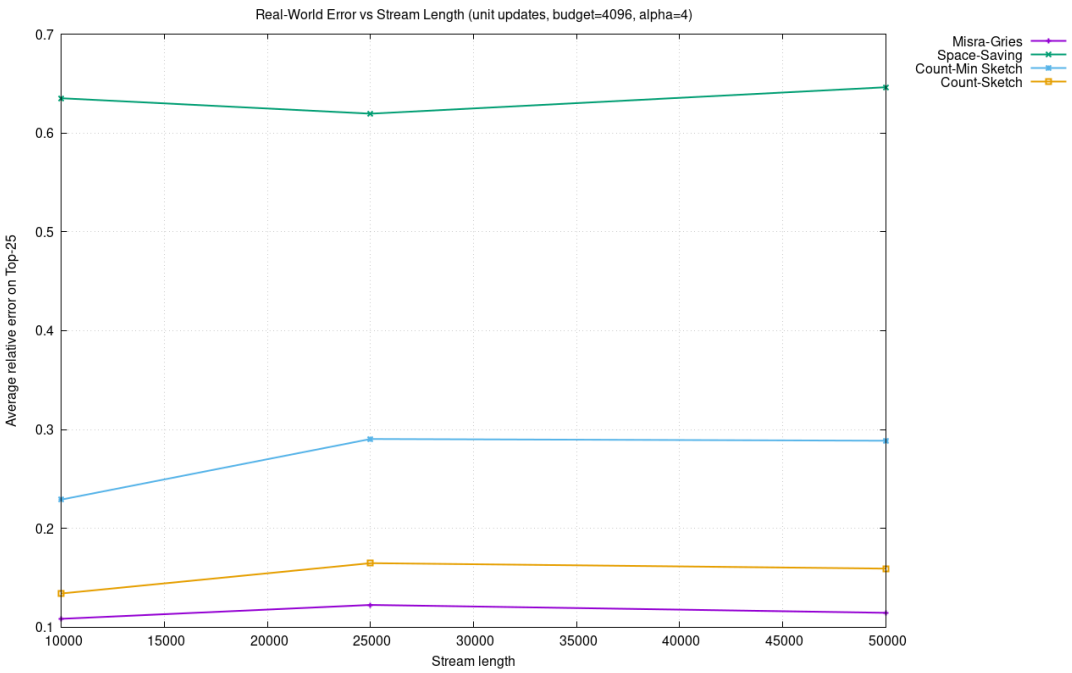

Figure 5: Unit-update real-world accuracy as the update-event count changes at fixed 4096-byte budget and $\alpha = 4$.

Algorithm	$\alpha = 1.5$	$\alpha = 2$	$\alpha = 4$	$\alpha = 8$
Misra-Gries	0.1017	0.0669	0.1145	0.1389
Space-Saving	0.2932	0.3952	0.6463	0.8866
Count-Min Sketch	0.3002	0.3022	0.2886	0.2991
Count-Sketch	0.1568	0.1490	0.1593	0.1451

Table 2: Unit-update average relative error on the real-world network trace at fixed 4096-byte budget and update-event count 50,000.

Skewed synthetic data still rewards summary methods once enough memory is available. For both unit and weighted updates, Count-Sketch is strongest at low budgets, while Space-Saving dominates once the budget is large enough to stabilize the candidate set. In unit mode, Count-Sketch gives 0.3933, 0.1344, and 0.1005 at 1KB, 2KB, and 4KB, while Space-Saving drops to 0.0002 at 8KB and 0.0000 at 16KB. The weighted-update pattern is similar: Count-Sketch remains best through 4KB with 0.3990, 0.1499, and 0.0928, and Space-Saving again reaches exact recovery at 8KB and 16KB. Weighted updates mostly hurt Space-Saving at small and medium budgets rather than changing the best high-budget choice.

The network trace gives the clearest practical error-space tradeoff. On the real-world dataset with $\alpha = 4$ and update-event count 50,000, both update modes show the same regime split. Count-Sketch is best at 1KB and 2KB: 0.4970 and 0.2066 in unit mode, versus 0.5119 and 0.2212 in weighted mode. At 4KB, Misra-Gries becomes best, with 0.1145 in unit mode and 0.1272 in weighted mode. At 8KB and 16KB, Space-Saving becomes dominant in both modes, dropping to 0.0028 and 0.0000 for unit updates and 0.0025 and 0.0000 for weighted updates. This is the most direct empirical instance of the requested tradeoff:

- Count-Sketch is the most robust method under severe memory pressure,
- Misra-Gries is strongest in the middle of the budget range,

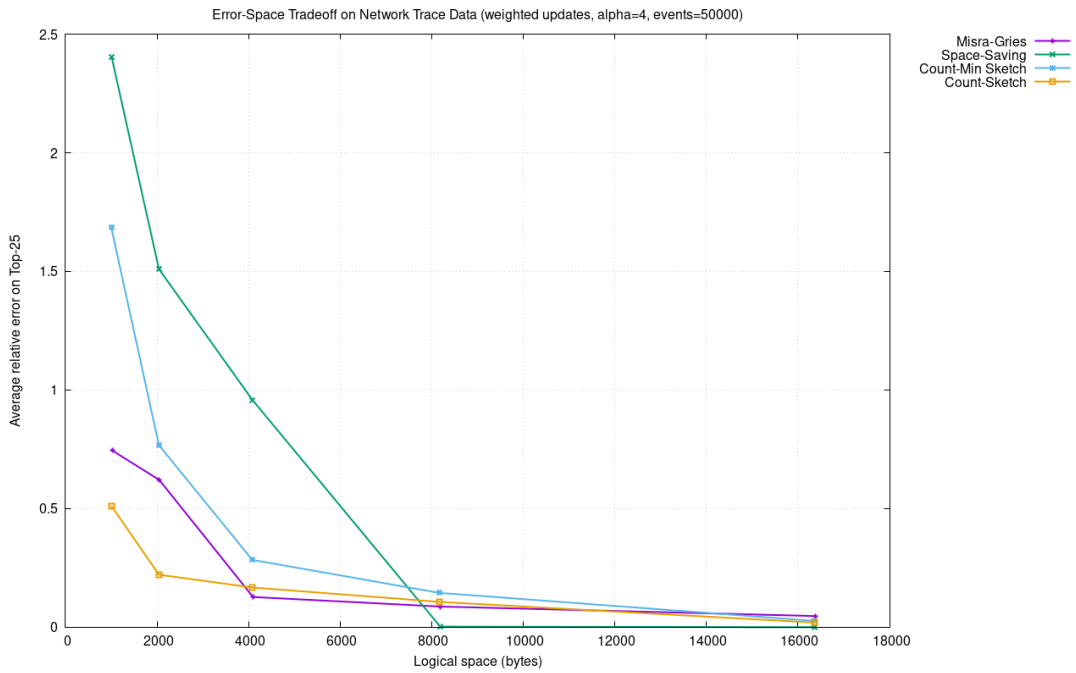

Figure 6: Weighted-update error-space tradeoff on the network-trace dataset for $\alpha = 4$ and update-event count 50,000.

Algorithm	1KB	2KB	4KB	8KB	16KB
Misra-Gries	0.7444	0.6207	0.1272	0.0868	0.0468
Space-Saving	2.4029	1.5097	0.9586	0.0025	0.0000
Count-Min Sketch	1.6846	0.7672	0.2836	0.1446	0.0265
Count-Sketch	0.5119	0.2212	0.1671	0.1066	0.0193

Table 3: Weighted-update average relative error on the real-world network trace for $\alpha = 4$ and update-event count 50,000 across the memory-budget sweep.

- Space-Saving wins once memory is large enough for a stable heavy-hitter summary.

Weighted updates mainly amplify medium-budget replacement error. The weighted track uses the same event-count grid but larger realized masses. On the real-world dataset at 50,000 events and $\alpha = 4$, the average realized masses are approximately $I = 66{,}617$, $D = 39{,}970$, and $I - D = 26{,}647$, compared with $I = 31{,}250$, $D = 18{,}750$, and $I - D = 12{,}500$ in unit mode. Despite this larger scale, the sketches remain stable: Count-Min Sketch changes only from 0.2886 to 0.2836 at 4KB, and Count-Sketch changes from 0.1593 to 0.1671. The main degradation is in Space-Saving, whose 4KB error rises from 0.6463 to 0.9586, showing that weighted updates make replacement mistakes more costly when the budget is only moderate.

Higher α hurts Space-Saving far more than the sketches. At a fixed 4096-byte budget on the real-world dataset, Misra-Gries remains the best method across all four α values in both modes. In unit mode it ranges from 0.0669 to 0.1389, and in weighted mode from 0.0733 to 0.1665. Count-Sketch stays comparatively stable: 0.1451–0.1593 in unit mode and 0.1395–0.1671 in weighted mode. Count-Min Sketch is also stable, staying near 0.29 in unit mode and improving slightly from 0.2977 to 0.2635 in weighted mode as α increases. Space-Saving is the clear outlier: it grows from 0.2932 to 0.8866 in unit mode and from 0.1793 to 1.9269 in weighted mode. This confirms that high deletion intensity is most damaging to replacement-

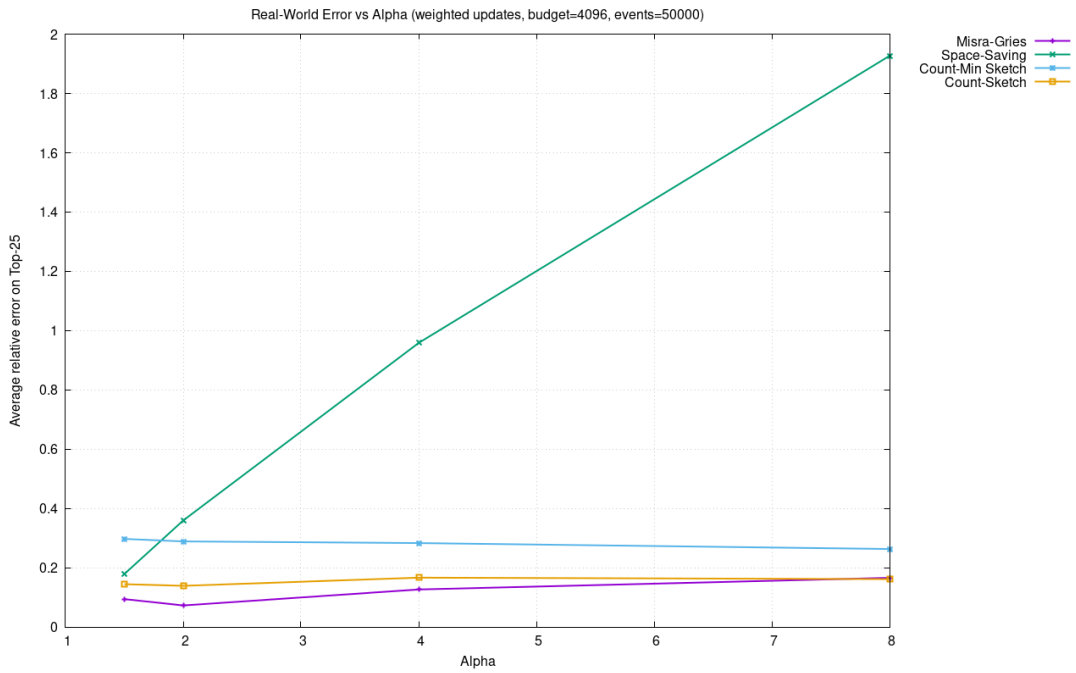

Figure 7: Weighted-update real-world accuracy as α changes at fixed 4096-byte budget and update-event count 50,000.

Algorithm	$\alpha = 1.5$	$\alpha = 2$	$\alpha = 4$	$\alpha = 8$
Misra-Gries	0.0946	0.0733	0.1272	0.1665
Space-Saving	0.1793	0.3599	0.9586	1.9269
Count-Min Sketch	0.2977	0.2892	0.2836	0.2635
Count-Sketch	0.1450	0.1395	0.1671	0.1620

Table 4: Weighted-update average relative error on the real-world network trace at fixed 4096-byte budget and update-event count 50,000.

based summaries.

Update-event count matters less than memory budget. At fixed 4096 bytes and $\alpha = 4$ on the real-world dataset, the unit-update errors vary only moderately as the event count increases from 10,000 to 50,000: Misra-Gries moves from 0.1085 to 0.1145, Count-Sketch from 0.1341 to 0.1593, and Count-Min Sketch from 0.2291 to 0.2886. The weighted-update pattern is similar for the sketches and Misra-Gries, although Space-Saving stays much worse at this budget, between 0.8185 and 0.9586. The dominant empirical effect in this study is therefore memory budget rather than moderate growth in the number of updates.

6 Advanced Design Proposal

The most promising next step is an **budget-adaptive hybrid estimator**. The proposed design combines:

1. a Count-Sketch front-end for low-budget robustness under arbitrary churn,
2. a Space-Saving candidate cache that activates only for items whose Count-Sketch estimate exceeds a promotion threshold,
3. a small exact deletion ledger for promoted items.

The motivation comes directly from the measured tradeoffs:

- Count-Sketch is the most reliable method at the smallest budgets,
- Space-Saving is the strongest high-memory heavy-hitter tracker on the skewed and real-world workloads,
- Misra-Gries remains a useful middle-budget fallback,
- the weighted-update experiments show that replacement mistakes become much more expensive at medium budgets.

An adaptive controller could start with a Count-Sketch-heavy allocation when the budget is tight or α is large, then move memory toward the promoted candidate cache as empirical skew becomes clear. This should improve the Pareto frontier by combining the low-budget stability of Count-Sketch with the high-budget precision of Space-Saving.

7 Conclusion

The project now satisfies all required tasks with the user-provided network dataset as the real-world workload. The implementations support strict turnstile streams with α-bounded deletions, the experiments cover update-event count, α, and multiple memory budgets, and the report now includes both unit-update and weighted-update evaluations together with explicit error-space guarantees in terms of $I - D$.

The main empirical takeaway is that there is no single universally best method:

- Count-Sketch is strongest under severe memory pressure,
- Misra-Gries is strongest in the middle of the budget range on the real-world trace,
- Space-Saving dominates the high-memory regime on skewed and real-world traffic,
- Count-Min Sketch improves substantially with space and becomes competitive at large budgets, especially on balanced data,
- weighted updates largely preserve these rankings but expose a sharper medium-budget penalty for Space-Saving.